with a `using` statement. This disposes of any `IDisposable` services that were created by the scoped container and prevents memory leaks.

The following listing shows a version of the `ExchangeRatesHostedService` that stores the latest exchange rates as an EF Core entity in the database. It creates a new scope for each iteration of the `while` loop and retrieves the scoped `AppDbContext` from the scoped container.

Listing 34.5 Consuming scoped services from an `IHostedService`

```
public class ExchangeRatesHostedService : BackgroundService           ❶
{
    private readonly IServiceProvider _provider;                       ❷
    public ExchangeRatesHostedService(IServiceProvider provider)      ❷
    {
        _provider = provider;
    }

    protected override async Task ExecuteAsync(
        CancellationToken stoppingToken)
    {
        while (!stoppingToken.IsCancellationRequested)
        {
            using(IServiceScope scope = _provider.CreateScope())      ❸
            {
                var scopedProvider = scope.ServiceProvider;            ❹

                var client = scope.ServiceProvider                     ❺
                    .GetRequiredService<ExchangeRatesClient>();        ❺

                var context = scope.ServiceProvider                    ❻
                    .GetRequiredService<AppDbContext>();               ❻

                var rates = await client.GetLatestRatesAsync();        ❻

                context.Add(rates);                                    ❻
                await context.SaveChanges(rates);                      ❻
             }                                                         ❼

            await Task.Delay(TimeSpan.FromMinutes(5), stoppingToken); ❽
```